Plano de Testes Exaustivo – Teixeira Tratores

1. Cadastro de Cliente

1.1 Testes de Validação de Campos

- Nome: Válido (3–100 caracteres). Inválido: vazio, <3, >100, caracteres especiais.
- CPF: Válido formato e algoritmo; Inválido: formato incorreto, sequência repetida, checksum errado, vazio.
- Endereço: Válido (≥5 caracteres); Inválido: vazio, <5 caracteres.
- Telefone: Válido (10–11 dígitos + máscara); Inválido: dígitos insuficientes, letras, vazio.

1.2 Cenários de Fluxo

- Fluxo Sucesso: preenchimento correto → POST /api/clientes = 201 → toast sucesso → redireciona.
- Erro Frontend: campos obrigatórios faltando → erros inline + bloqueio de submit.
- Erro Backend 400: mensagens por campo conforme resposta JSON.
- Erro Rede/500: toast 'Falha de conexão', mantém form, botão reabilita.
- Concorrência: dupla submissão mesmo CPF → 409 → toast 'Cliente já existe'.

1.3 Testes de UI/UX

- Toasts topo-direito, dismiss após 3s.
- Erros inline com aria-describedby e foco no primeiro erro.
- Botão 'Salvar' com aria-busy durante envio.
- Navegação via teclado com ordem lógica.

1.4 Integração com Firebase

- Firestore: escrita em 'clientes'; rules negam sem auth (403).
- Realtime DB: nova entrada aparece ao vivo; desconexão exibe status.

1.5 Segurança

- Proteção de rota /clientes/new; redireciona se não auth.
- Bypass cookie httpOnly negado no servidor.
- Sanitização de inputs contra XSS (<script> escapado).

1.6 Edge Cases e Concurrency

- CPF 000.000.000-00 inválido.
- Nome 100 chars válido; 101 inválido.
- Caracteres unicode em telefone inválidos.

2. Cadastro de Funcionário

2.1 Validação de Campos

- Nome: mesmas regras do cliente.
- E-mail: válido com '@domínio'; inválido sem '@', sem domínio, espaços.
- Cargo: opções admin, manager, employee; obrigatório.
- CPF, Telefone, Endereço: mesmas regras do cliente.

2.2 Fluxos e Permissões

- Acesso restrito a admins; 403 + toast 'Sem permissão'.
- Sucesso: grava convite em 'convites_funcionarios', status pending, toast sucesso.
- Falha Invite 409: inline 'Convite já enviado'.
- Erro rede/500: toast 'Erro no servidor'.

2.3 UI/UX

- Botão desabilita após clique até resposta.
- Status pending aparece em listagem em tempo real.
- Mensagens inline + toast conforme fluxo.

2.4 Integração Firebase

- Auth: registra convite, não cria conta.
- Firestore rules negam sem admin (403).
- Race: dois invites same email → segundo 409.

2.5 Segurança

- Middleware Next.js protege rota.
- Curl direto sem admin → acesso negado.

2.6 Edge Cases

- E-mail mistas cases normalizadas; duplicatas detectadas.
- Cargo inválido no payload → 400.

3. Cadastro de Fornecedor

3.1 Validação de Campos

- Nome: 3–100 chars obrigatórios.
- CNPJ: formato + algoritmo; inválido: menos dígitos, checksum, vazio.
- Endereço: ≥5 chars; obrigatório.
- Telefone: mesmas regras do cliente.
- Descrição: ≥20 chars; obrigatório.

3.2 Fluxos

- Sucesso: grava em 'fornecedores', toast sucesso.

- CNPJ duplicado 409 → inline 'CNPJ já cadastrado'.
- Erro rede → toast 'Erro de conexão'.

3.3 UI/UX

- Inline errors, spinner, botão desabilita.
- Form mantém dados após erro.

3.4 Integração & Segurança

- Firestore rules: admin/manager.
- Proteção de rota.

3.5 Edge Cases

- CNPJ sem máscara → limpa e valida.
- Descrição com 1000 chars → definir máximo.

4. Cadastro de Projeto

4.1 Validação de Campos

- Nome: 3–100 chars obrigatório.
- Orçamento: número >0; inválido ≤0 ou não-numérico.
- Prazo: data ≥ hoje; inválido < hoje.
- Descrição: ≥20 chars.
- Cliente: select obrigatório, existência validada.

4.2 Fluxos

- Sucesso: grava em 'projetos', toast sucesso.
- Cliente inexistente 400 → inline.
- Erro rede/perm negada.

4.3 UI/UX

- Submit desabilitado até selects carregarem.
- Skeleton loader para lista de clientes.

4.4 Integração / Edge / Segurança

- Realtime updates no dashboard.
- Simular desconexão/reconexão.

5. Cadastro de Conta a Pagar/Receber

5.1 Testes de Validação de Campos

- Nome: obrigatório, 3–100 caracteres; inválido: vazio, <3 ou >100, caracteres especiais.
- Valor: obrigatório, número > 0; inválido: 0, número negativo, texto, valor com mais de 2 casas decimais.
- Data de Vencimento: obrigatório, formato YYYY-MM-DD; inválido: data passada, formato incorreto, valor vazio.
- Método de Pagamento (select): obrigatório, opções pré-definidas (ex: 'Boleto', 'Cartão', 'Pix'); inválido: valor fora da lista, vazio.
- Status de Pagamento (select): obrigatório, opções ('Pendente', 'Pago'); inválido: vazio, valor não reconhecido.
- Descrição: obrigatório, mínimo 20 caracteres; inválido: <20, excesso (>1000), vazio.
- Fornecedor/Projeto (select): obrigatório conforme contexto; inválido: vazio, ID inexistente.

5.2 Cenários de Fluxo

- Sucesso: todos os campos válidos → POST /api/contas = 201 → toast 'Conta cadastrada com sucesso' → redireciona para listagem.
- Falha Frontend: tente submeter com campo vazio → form não submete, erros inline exibidos em cada campo faltante.
- Falha Backend 400: backend retorna JSON com erros por campo (ex: valor inválido) → exibir erros inline.
- Falha Backend 409: conta duplicada (mesmo nome e data) → exibir toast 'Conta já existe neste período'.
- Erro Rede/500: timeout ou erro servidor → exibir toast 'Erro de conexão. Tente novamente.'; formulário mantém dados e botão reabilita.
- Permissão Negada (403): usuário sem permissão (não admin/manager) tenta acessar endpoint → redirecionar para página de erro 403 ou toast 'Sem permissão'.
- Concorrência: duas submissões simultâneas do mesmo registro → primeira aceita, segunda retorna 409 → toast 'Conta já existe'.

5.3 Testes de UI/UX

- Botão 'Salvar' desabilita ao clicar e mostra spinner até resposta.
- Toasts de sucesso/erro aparecem e desaparecem após 3 segundos, com aria-live.
- Erros inline associados a cada campo via aria-describedby e foco no primeiro erro.
- Formulário mantém valores após erro de rede ou backend.
- Tab-navegação percorre campos na ordem correta; labels visíveis claros.

5.4 Integração com Firebase

- Firestore: criação de documento na coleção 'contas'; testar rules negando escrita sem auth (403).
- Firestore: testes de regras negando criação sem permissão de papel (admin/manager).

- Realtime Database: se aplicável, nova conta aparece em tempo real na listagem; simular goOffline/goOnline e verificar fallback.
- Simular latência: debugar delayed writes e UI indica 'sincronizando'.

5.5 Segurança

- Proteção de rota '/contas/new': sem sessão → redireciona para login.
- Verificar CSRF: requisições POST sem token correto são rejeitadas.
- Teste XSS: inserir scripts em campos de texto e validar que são escapados.
- Verificar headers de segurança (CSP, X-Frame-Options).

5.6 Edge Cases e Race Conditions

- Valor extremo: muito alto (ex: 10 milhões) e muito baixo (>0.01); verificar aceitação.
- Datas limite: hoje e ano muito futuro; aceitar hoje, recusar ano passado ou muito adiante.
- Selects com opções dinâmicas: link para fornecedor removido no meio do preenchimento → tratamento de erro.
- Simular múltiplas abas: atualizar dados simultaneamente e detectar duplicidade via 409.

6. Autenticação e Sessão

6.1 Validação de Campos

- E-mail: obrigatório, formato válido (XXX@YYY.ZZZ); inválido sem '@', sem domínio, com espaços.
- Senha: obrigatória, mínimo 8 caracteres, ao menos 1 maiúscula, 1 número, 1 caracter especial; inválido se faltar regras.

6.2 Cenários de Fluxo

- Sucesso: e-mail e senha corretos → login → cookie httpOnly criado → redireciona dashboard.
- Falha Frontend: campo vazio → bloqueio submit + erro inline.
- Erro Backend Auth/wrong-password → toast 'E-mail ou senha incorretos'.
- Erro Rede/Timeout → toast 'Falha na conexão'.
- Permissão: rota /dashboard acessível apenas se cookie válido; caso contrário, redirecionar login.

6.3 UI/UX

- Botão login desabilitado até validação front-end OK.
- Spinner em botão durante requisição.
- Erro inline abaixo de cada campo.
- Toasts no topo em falhas e sucesso de logout.

6.4 Integração Firebase

- Firebase Auth: testar respostas de erro code 'auth/user-not-found', 'auth/wrong-password', 'auth/too-many-requests'.
- Simular offline: offlineAuth.setPersistence falha → erro genérico.

6.5 Segurança

- Proteção de rota via middleware Next.js; acesso direto a /dashboard bloqueado se não auth.
- Teste de CSRF: meta tag _csrf ausente ou incorreta impede login.
- Teste de XSS: inputs escapam caracteres HTML.

6.6 Edge Cases e Concurrency

- Múltiplos logins simultâneos: cookies gerados e sessão única por dispositivo.
- Senha com 100 caracteres: aceita e funciona.

7. Cadastro de Usuário (Signup)

7.1 Validação de Campos

- E-mail convite: obrigatório, formato válido, deve existir convite pendente no Firestore.
- Senha: obrigatório, mesmas regras de login.
- Confirmação de senha: deve igualar senha; inválido se diferente.

7.2 Cenários de Fluxo

- Sucesso: convite válido + senha forte → cria usuário Auth + documento Firestore + marca convite usado → toast sucesso + redireciona login.
- Falha Frontend: senha e confirmação diferentes → erro inline.
- Erro Backend: convite não encontrado → toast 'Convite inválido ou expirado'.
- Erro Rede: toast genérico.
- Tentativa de reuso de convite já usado → 409 → inline convite.

7.3 UI/UX

- Botão 'Criar conta' desabilitado até todos campos válidos.
- Spinner durante submit.
- Erros inline nos campos e toast geral.

7.4 Integração Firebase

- Firestore transaction: marcar convite usado ou rollback em falha.

7.5 Segurança

- URL de signup acessível apenas com token de convite; sem token, redireciona para erro 404.
- Proteção contra CSRF e XSS nos inputs.

7.6 Edge Cases e Concurrency

- Duas tentativas simultâneas usar mesmo convite: somente primeiro deve funcionar; segundo recebe erro.

8. Recuperação de Senha

8.1 Validação de Campos

- E-mail: obrigatório, formato válido, deve existir usuário.
- Código de recuperação: obrigatório, numérico, 6 dígitos.
- Nova senha: mesmas regras de senha.
- Confirmação: deve casar.

8.2 Cenários de Fluxo

- Solicitação: enviar e-mail → cria documento recovery no Firestore com status pendente → toast 'Aguarde aprovação'.
- Aprovação Admin: status pendente → admin aprova → e-mail recebe código.
- Redefinição: inserir código válido + nova senha → atualiza senha Auth e remove documento → toast sucesso.
- Falha Frontend: campos vazios → inline.
- Erro Backend: código inválido/expirado → toast 'Código inválido ou expirado'.
- Erro Rede: toast genérico.
- Múltiplos pedidos pendentes: impedido pelo backend → 429.

8.3 UI/UX

- Etapas em um wizard com step indicator.
- Botões desabilitados até validação das etapas.
- Toasts e erros inline por etapa.

8.4 Integração Firebase

- Firestore: criar/leitura no recoveryRequests; testar rules e erros 403.
- Auth.sendPasswordResetEmail tested and errors 'auth/user-not-found'.

8.5 Segurança

- Código de recuperação com expiração de 15min; test expiry.
- CSRF protection nas requisições.

8.6 Edge Cases e Concurrency

- Dois pedidos simultâneos: segundo recebe erro 429.
- Reutilização de código após uso → negado.

9. SSE / Realtime

9.1 Validação de Campos

- Não há campos; foco em eventos recebidos.

9.2 Cenários de Fluxo

- Conexão: ao montar componente, conectar EventSource → receber evento 'update' → atualizar UI.
- Falha Frontend: EventSource não suportado → exibir fallback de polling.
- Falha Backend: mensagem malformada → ignorar e log error.
- Permissão: se JWT expirado, server fecha conexão → reconectar com token refrescado.

9.3 UI/UX

- Carregamento inicial mostra skeleton.
- Indicator de online/offline do SSE.
- Atualização suave de lista sem full refresh.

9.4 Integração Firebase

- Testar Realtime Database listeners on('value') e off() no unmount.
- Simular goOffline/goOnline e verificar UI.

9.5 Segurança

- Verificar que EventSource não expõe token em URL pública.
- Proteger rota SSE via JWT no header ou cookie.

9.6 Edge Cases e Concurrency

- Múltiplas instâncias do listener montadas/desmontadas não duplicam eventos.
- Reconexão exponencial backoff testato.

10. Segurança Geral

10.1 Testes de Fluxo

- Acesso não autenticado a qualquer rota /app/* → redireciona login.
- Injeção XSS: tentar `<script>` em todos campos → alerta bloqueado.
- CSRF: requisições POST sem token validadas no servidor rejeitadas.

10.2 Testes de Concurrency

- Session hijacking tentado trocando cookies → acesso negado.
- Replay attack: requisição repetida com mesmo nonce → rejeitada.

10.3 Exemplos de Pentest

- Usar OWASP ZAP para scan XSS/CSRF.

11. Usabilidade e Acessibilidade

11.1 Testes de UX

- Navegação completa por teclado (Tab/Shift+Tab) cobre todos elementos.
- Labels visíveis e programaticamente vinculados aos inputs.
- Contraste de cores WCAG AA: verificar em pages de formulário.

11.2 Testes de ARIA

- Aria-live nos toasts para leitores de tela.
- Foco gerenciado após erro: move para primeiro erro.

12. Integração E2E Completa

12.1 Fluxo Principal

- Cadastro cliente → login → CRUD cliente → logout.
- Cadastro funcionário via admin → login funcionário → CRUD funcionário.
- Signup via convite → login → teste permissão.

12.2 Cenários de Falha

- Simular backend 500 em meio ao fluxo → usuário retorna ao passo anterior com erro.